Auditing an income-prediction model for sex bias

Supervised Learning & Model Engineering (MSA-DATB09-04): project report

Manel Rahali, Soumaya Atoui, Hadiqa Javaid

Albert School, Fall 2026

Repository: https://github.com/mrahali-cell/Supervised-Learning-Model-Engineering

Dataset: Option 1, Adult / Census Income

`fetch_openml(name="adult", version=2)`, 48,842 people, 15 columns, 1994 US Census [9].

Abstract

A company sells models that predict whether a person earns more than $50K a year, and its clients use the predictions in decisions such as audits. We built the best model that six legitimate inputs allow, then measured how differently it treats women and men, although it never sees sex. Logistic regression, a random forest and LightGBM all stop at about 0.71 PR-AUC, so the limit is the information in the inputs, not the model. The final model, a tuned LightGBM, scores 0.7148 PR-AUC and 0.8945 ROC-AUC on the untouched test set, and its probabilities are calibrated for both sexes. At its default threshold it still finds 57% of high-earning women against 78.5% of high-earning men, mostly through marital status. Per-sex thresholds equalise the two error rates on the test set for about 0.03 of balanced accuracy, but they widen the gap in precision and still fail the four-fifths rule.

1 Problem, metrics and protocol

Use case. If an income model treats some groups worse than others, the audit decisions built on it become unfair, and the company's clients face sanctions, legal action and financial loss. We act as the auditor: build the most accurate honest model, measure how unequally it treats women and men, reduce part of the gap and state what that costs. Our hypothesis is that a model that never sees sex rebuilds the gender gap through proxies such as marital status, occupation and hours worked.

Target and metrics. The target `high_income` equals 1 when income exceeds $50K (23.94% of people). In an audit neither error dominates, so we took the rare class as positive, which makes the ranking metric informative. Accuracy is not: always answering "no" scores 76%. We rank models with the average precision (PR-AUC),

$$\mathrm{AP} = \sum_k (R_k - R_{k-1})\, P_k, \tag{1}$$

where P_k and R_k are the precision and recall at the k-th threshold. A random ranking scores the prevalence, 0.239. ROC-AUC is reported beside it, and the quality of the probabilities is judged with the Brier score and the calibration error (Section 6).

Fairness definition, fixed before any result. Our primary definition is **equalized odds**: equal true- and false-positive rates for women and men [6], because a general-purpose model can harm someone through both kinds of error. The legal screen is the **four-fifths rule** on selection rates [5]. Demographic parity and predictive parity are reported too (Section 8).

Honest protocol. After fixed cleaning rules, the data was split once into a development set (80%, 39,032 people) and a test set (20%, 9,758 people), stratified on the target with `random_state=42`. Every choice (features, models, hyperparameters, threshold, calibration, mitigation) was made with 5-fold stratified cross-validation on the development set. Everything that learns from the data sits inside one scikit-learn `Pipeline` [11] and is refitted in every fold. The test set was opened once, at the end, with every choice frozen.

2 Data, leakage and features

Integrity and cleaning. Three columns have missing values: `workclass` (5.73%), `occupation` (5.75%) and `native-country` (1.75%). They are not random: the same 2,799 people miss both `workclass` and `occupation`, and the 10 extra missing occupations belong to people who never worked, for whom occupation does not apply. We removed 52 exact duplicates before the split so that no record sits on both sides. Extreme values are census caps (244 capital gains at 99,999, 137 people at 99 hours, 55 at age 90), so they were kept. Categories with a few dozen people were merged with the closest category by meaning and income rate. These rules learn nothing from the data, so they are safe before the split.

Leakage. For each column we asked whether it would be known when a prediction is made. Capital gains fail this test: 98.4% of people with a gain of at least $10,000 earn more than $50K, and all 244 at the cap do. The variable is most likely derived from income, so we dropped `capital-gain` and `capital-loss`. Our scores are therefore not comparable with published ones (0.92–0.93 ROC-AUC), which use them [3].

Inputs and sensitive attributes. We also dropped `education` (a copy of `education-num`), `fnlwgt` (a sampling weight) and `relationship`, which reveals sex through Husband and Wife. `sex`, `race` and `native-country` are kept for the audit only. Six inputs remain: `age`, `workclass`, `education-num`, `marital-status`, `occupation` and `hours-per-week`.

Where the gender gap lives. 30.4% of men and 10.9% of women earn more than $50K. Among married people the rates are equal (44.8% of women, 44.6% of men), but 60.9% of men are married against 15.3% of women (Figure 1). Women also work fewer hours (36.4 against 42.4 on average) and in different occupations. Leaving `sex` out therefore does not make a model blind to it: its inputs are proxies [2]. These are associations, not causes.

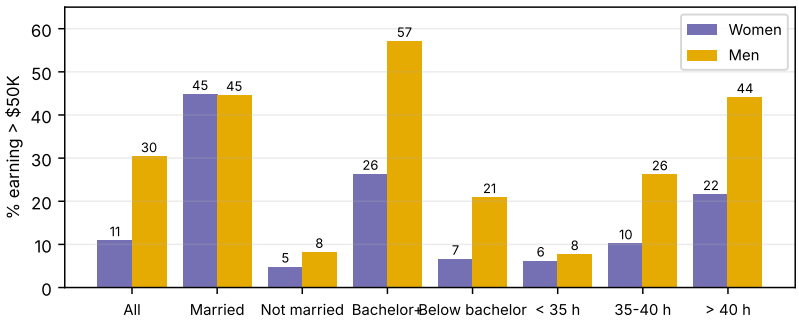


Figure 1: Share earning more than $50K, women and men, overall and within groups (development set). Married women and men earn alike; the overall gap comes largely from who is married, plus gaps within education and hours groups.

Feature engineering. A linear model cannot draw a curve or a regime from one column, so we added five fixed rules, each backed by a pattern in the development set (share earning at most $50K): `is_married` (55.5% against 93.6% for everyone else); `has_degree`, `education-num` ≥ 13 (52.0% against 83.9%); `part_time` (< 35 h, about 93%) and `overtime` (> 40 h, 56–65%, against 78.7% at 40 h), a signal hidden by both medians being 40 h; and `age_squared` for the rise and fall with age (98.9% at 24 or younger, 60.6% at 45–54, 79.6% at 65 and over).

The Pipeline. `build_pipeline(model)` chains the feature rules, a `ColumnTransformer` and the model. Numbers are imputed with the training median and standardised; categories get an explicit "Unknown" level, which keeps the fact that the value was missing, then one-hot encoding with `handle_unknown="ignore"`; the binary features pass through. Six inputs become 37 columns. The engineered features raise logistic

regression from 0.6880 to 0.7082 PR-AUC and leave LightGBM unchanged (0.7155), since trees find these patterns by themselves.

3 Class imbalance

The positive class is 23.9% of people. Beyond choosing PR-AUC, we tested re-weighting as a hyperparameter of each model: `class_weight="balanced"` for logistic regression and the random forest, `scale_pos_weight` for LightGBM. Weights were part of the first Optuna search spaces (`tuning/tune.py`, git tag `baseline-v1`) and were then isolated with one validation curve per model. Re-weighting never improved the ranking and always degraded the probabilities (Table 1). This is what theory predicts: if every positive gets weight w, the best-fitting probability becomes

$$p_w = \frac{w\,p}{w\,p + 1 - p}, \tag{2}$$

an increasing function of the true probability p. People keep their order, so the ranking metrics do not move, but every probability is pushed up. The first tuning confirms it: Optuna chose no weights for logistic regression and a mild one for LightGBM (1.32), while the forest that chose balanced weights had the worst probabilities of the three (Brier 0.1365). We therefore do not re-weight. The imbalance is handled at the decision step, with a threshold below 0.5 chosen on the development set (0.343, the best-F1 point, Section 5). We did not try SMOTE: half of our inputs are categorical, and interpolating between two people's occupations has no meaning.

Model (5-fold CV, other settings at defaults)	Weighting	PR-AUC	Brier
Logistic regression ($C = 1$)	none / balanced	0.7083 / 0.7063	0.1113 / 0.1397
Random forest (200 trees)	none / balanced	0.6421 / 0.6282	0.1303 / 0.1446
LightGBM	`scale_pos_weight` 1 / 3.2	0.7155 / 0.7160	0.1101 / 0.1378

Table 1: Re-weighting the positive class leaves the ranking unchanged or worse and damages the probabilities. 3.2 is the ratio of negatives to positives.

4 Models, diagnosis and tuning

We compared three families of increasing flexibility on the same folds and metrics. For each one we first diagnosed overfitting or underfitting from the gap between training and validation scores, then drew validation curves (one setting changes, the others stay fixed), and only then tuned.

Logistic regression. The model predicts $\hat{p} = \sigma(w^\top x + b)$ with $\sigma(z) = 1/(1+e^{-z})$, and training minimises the penalised log-loss

$$C \sum_{i=1}^{n} \Big[-y_i \log \hat{p}_i - (1-y_i)\log(1-\hat{p}_i) \Big] + \Omega(w), \qquad \Omega(w) = \tfrac{1}{2}\|w\|_2^2 \text{ (L2) or } \|w\|_1 \text{ (L1)}, \tag{3}$$

so $C = 1/\lambda$: a small C is a strong penalty. The gap between training and validation PR-AUC is 0.0014, about ten times smaller than the fold-to-fold noise, so the model does not overfit. Its training score is itself only 0.710: a small gap with a mediocre score is underfitting (bias). The validation curves agree (Figure 2a): from $C = 0.3$ to 100 the score is flat at 0.708 for both penalties, while a strong penalty only adds bias ($C = 0.01$: 0.698 with L2, 0.682 with L1; L1 at $C = 10^{-4}$ sets every weight to zero and becomes the Dummy). Elastic Net never beat pure L2. Since regularisation can only add bias, the remedy is new information: the engineered features added 0.020. Among three interactions tested with a rule fixed beforehand (a gain of at least 0.001 in at least 4 of 5 folds), only marital status $\times$ hours passed (+0.0016, better in all 5 folds). We did not add it, because it combines two proxies for sex.

Random forest. Each tree splits greedily on the impurity of its nodes, and a fully grown tree memorises: one tree reaches 0.977 PR-AUC on its training folds and 0.444 on validation. Averaging 100 such trees (scikit-learn defaults) gives 0.639, still with a gap of 0.333, because the trees remain fully grown. Limiting growth is what fixes it (Figure 2b): with at least 10 people per leaf, validation reaches 0.714 with a gap of 0.051; at 100 people per leaf the gap almost closes (0.008) but validation drops to 0.705, the start of underfitting. Depth tells the same story (depth 4: 0.684 with a gap of 0.004). Smaller bootstrap samples also help (30% of people per tree: 0.685 against 0.642), while more than 200 trees changes nothing.

LightGBM. Boosting adds trees one after another, each correcting the current errors: $F_m(x) = F_{m-1}(x) + \eta\, h_m(x)$, with learning rate η [7]. The rate and the number of trees therefore act together (Figure 2c). The best validation scores sit where η times the number of trees is about 5 to 8 (0.7170 at $\eta = 0.03$ with 200 trees, 0.7163 at $\eta = 0.01$ with 800), and more steps overfit (0.672 at $\eta = 0.1$ with 1,600 trees, gap 0.244). Unlike the forest, boosting can overfit by adding trees. The defaults already sit near the top (0.7155, gap 0.042): mild variance.

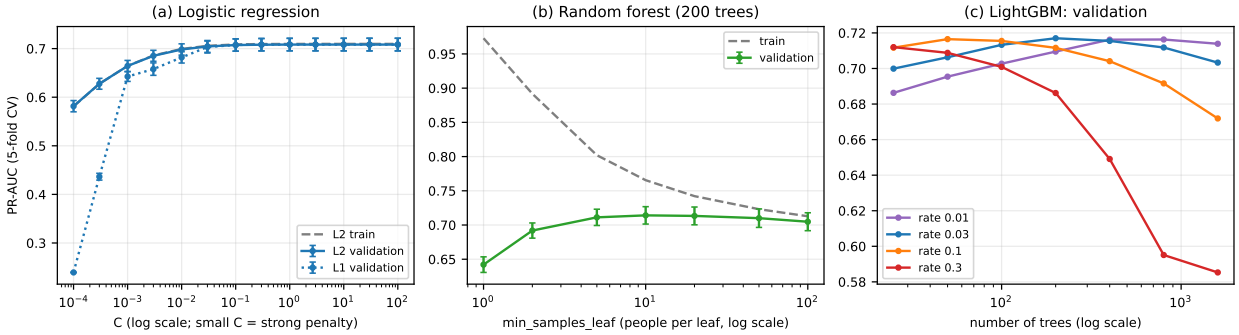


Figure 2: Validation curves, 5-fold CV on the development set; error bars are fold standard deviations. (a) Logistic regression: the training curve lies on the validation curve (no variance); a strong penalty underfits. (b) Random forest: from memorising (leaf 1) to underfitting (leaf 100). (c) LightGBM: too many boosting steps overfit, sooner at higher rates.

Tuning with Optuna. Each model was tuned in one Optuna study [1]: TPE sampler, seed 42, 25 trials, objective the mean 5-fold validation PR-AUC of the whole Pipeline. The first 10 trials are random, then TPE samples where earlier trials scored well. A first tuning (v1) used wide spaces that included class weights. For v2 we rebuilt each space from the validation curves: settings with no effect or a harmful one were fixed (no class weights for any model; 300 trees and Gini for the forest), and each searched range is the plateau of its curve plus a margin. The best v2 settings are listed in Appendix B.

	Development set, 5-fold CV					Test set, once		
	Untuned	Tuned v2						
Model	PR-AUC	Train	Val.	Gap	Brier	PR-AUC [95% CI]	ROC-AUC	Brier
Dummy (prior)	0.2394	–	–	–	0.1821	0.2394	0.5000	0.1821
Logistic regression	0.7082	0.7097	0.7083	0.0014	0.1113	0.7041 [0.684, 0.724]	0.8895	0.1115
Random forest	0.7110	0.7536	0.7149	0.0386	0.1104	0.7140 [0.696, 0.733]	0.8926	0.1100
LightGBM (final)	0.7155	0.7341	0.7189	0.0152	0.1096	**0.7148** [0.696, 0.732]	**0.8945**	**0.1092**

Table 2: The model progression with the same metrics. Untuned: logistic regression with $C = 1$, random forest with 300 trees and 5 people per leaf, LightGBM defaults. The fold standard deviation of validation PR-AUC is 0.011–0.013 (population formula). Test intervals: 1,000 bootstrap resamples of people.

What each step bought. Logistic regression captures almost all the signal: 0.47 PR-AUC above the Dummy (Table 2). The forest and LightGBM add 0.003 and 0.007 untuned, less than the fold noise. Tuning v2 kept the ranking of v1 for all three models and left most of the gain elsewhere: smaller train-validation gaps and better

probabilities (forest Brier 0.1365 with v1's class weights against 0.1104; LightGBM 0.1112 against 0.1096). In the v2 studies of our tracking store, 22, 20 and 21 of the 25 trials (logistic regression, forest, LightGBM) landed within 0.002 of the best one (Appendix A), so once the space was right the search method mattered little. Three families of very different flexibility stopping at the same level point to the same conclusion: the ceiling is the information in six inputs, not the model. Re-running the forest's v2 study on another machine illustrates this plateau: the 10 random trials reproduced the same best score (0.7144), and the guided trials then reached 0.7146 with different settings (0.7149 in the original run).

Experiment tracking. Every study is logged in MLflow [14] as a parent run (model, version, budget, objective, CV scheme, best parameters and scores, optimisation-history figure) with one nested run per trial (hyperparameters, mean and standard deviation of PR-AUC and ROC-AUC, Brier score, per-fold metrics as an artifact). Calibration, fairness and SHAP runs log their metrics, figures and result files. Our tracking store holds 164 runs: 150 trials, 6 studies (v1 and v2 for three models) and 8 evaluation runs (Appendix A).

5 Final evaluation on the test set

Every choice was frozen on the development set: the v2 parameters, the decision thresholds, the calibrator and the mitigation. Each model was refitted on the 39,032 development people and the 9,758 test people were scored once. The test stage was run a second time only to redraw a figure title; every number was identical. The development score of LightGBM was slightly optimistic, as expected from choosing the best trial on the same folds: 0.7183 out-of-fold against 0.7148 on the test set. On the same test people, LightGBM beats logistic regression (PR-AUC $+0.0107$, 95% interval $[+0.0053, +0.0160]$; Brier -0.0023, $[-0.0030, -0.0014]$) and ties the random forest on ranking ($+0.0008$, $[-0.0038, +0.0050]$) while giving slightly better probabilities (Brier -0.0007, $[-0.0013, -0.00001]$). We keep **LightGBM v2** as the final model and logistic regression as the transparent benchmark. At the best-F1 threshold fixed on the development set (0.343), the final model reaches a precision of 0.604 and a recall of 0.752 on the test set (F1 0.670). A client with a different cost of errors would choose another threshold.

6 Calibration

A score is calibrated if, among people who receive about 0.3, about 30% are high earners. We measure it with the Brier score and the expected calibration error over 10 equal-frequency bins b:

$$\text{Brier} = \frac{1}{n}\sum_{i=1}^{n}(\hat{p}_i - y_i)^2, \qquad \text{ECE} = \sum_{b}\frac{n_b}{n}\left|\,\bar{y}_b - \bar{p}_b\,\right|, \tag{4}$$

where $\bar{y}_b$ and $\bar{p}_b$ are the observed rate and the mean prediction in bin b. The first tuning taught us what breaks calibration: with balanced class weights the random forest said 0.912 for people of whom 81.1% were high earners (ECE 0.132), and LightGBM with `scale_pos_weight` 1.32 was mildly over-confident (ECE 0.030). Without weights, the v2 models are calibrated as they come out of training (out-of-fold ECE: logistic regression 0.0059, forest 0.0109, LightGBM 0.0058). Platt scaling, $\hat{p}' = \sigma(A\,s + B)$ with s the log-odds score [12], changes nothing for LightGBM v2 (ECE 0.0057), and isotonic regression reaches 0.0010 but its step function creates ties and lowers PR-AUC from 0.7183 to 0.7135.

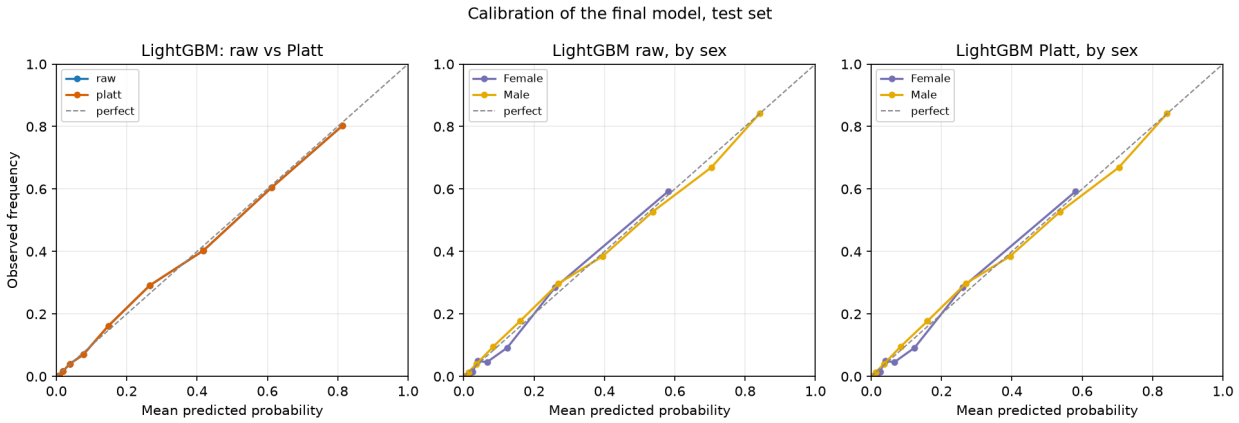


Figure 3: Reliability of the final model on the test set: raw and Platt probabilities (left), and by sex (centre: raw, right: Platt). Each point is a bin of people.

On the test set (Figure 3) the final model has a Brier score of 0.1092 and an ECE of 0.0091. Its mean prediction is 0.2401 for an observed rate of 0.2394; for women 0.113 against 0.109, for men 0.304 against 0.305. The curve shows no systematic over- or under-confidence: the top decile is predicted at 0.814 for 0.803 observed, and the highest-scored women are slightly under-predicted (0.581 against 0.592), deviations of the size of the noise in bins of about 326 women. We would trust these probabilities as they are for a downstream decision, for both sexes, without recalibration.

7 Explainability with SHAP

SHAP splits each prediction into one contribution per column [10]. For the final model the split is exact in log-odds:

$$f(x) = \phi_0 + \sum_j \phi_j(x), \tag{5}$$

where $f(x)$ is the score, ϕ_0 the average score and $\phi_j(x)$ the contribution of column j. Contributions add up, so the columns built from one input (its one-hot categories and its engineered features) can be summed into one contribution per input. We used TreeSHAP out-of-fold on the development set: each person is explained by the fold model that never saw them.

Global importance. The mean absolute contribution ranks marital status first (1.20 in log-odds), then age (0.74), education (0.52), hours (0.35), occupation (0.34) and work class (0.08). This makes business sense and matches the data analysis: marriage pools household income, age and education stand for experience and qualification, and hours and occupation set pay. No input with an implausible effect appears.

Where the score gap between women and men comes from. The mean score is -3.304 for women and -1.650 for men. Because SHAP is additive, the gap of -1.654 splits exactly into one part per input (Figure 4): marital status -1.089 (66%), hours -0.239 (14%), age -0.225 (14%), then occupation, education and work class (5% together). Education matters overall but hardly separates women from men. For the same value of an input the effect is about the same for both sexes, so the gap comes from the different values the two groups have, not from a different treatment of the same value. The test set gives the same split (marital status 65% of a gap of -1.671). This confirms our hypothesis on the model itself.

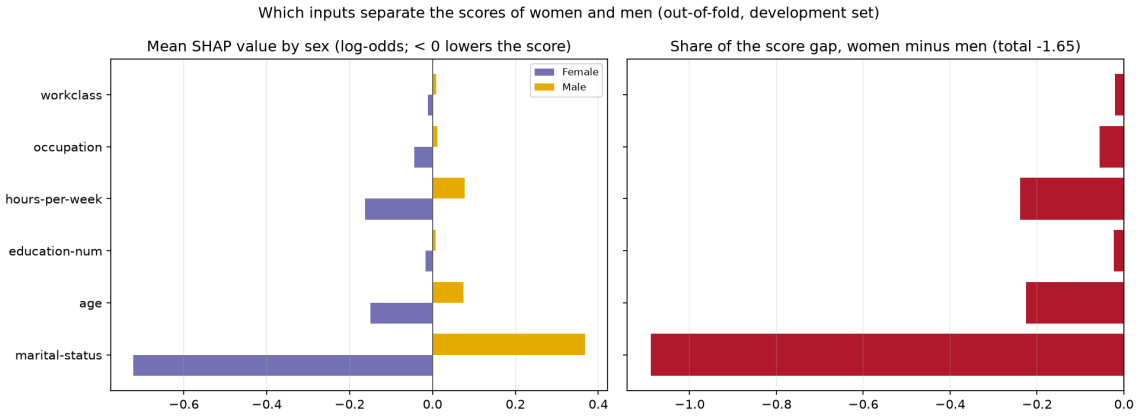


Figure 4: Mean SHAP contribution of each input for women and men (left) and its share of the score gap (right), out-of-fold, development set. The model never sees sex.

Local explanations. To avoid picking convenient examples, we explained the person with the median score in each of the four outcomes at the 0.343 threshold (Figure 5). The false positive is a married woman with 9 years of education in a managerial job, flagged at 0.51 mostly because she is married (+1.42). The false negative is a divorced woman in a managerial job who works 60 hours and earns more than \$50K; being divorced (−0.88) cancels her long hours (+0.88) and she is scored 0.20. In both errors marital status, the strongest proxy for sex, decides the outcome.

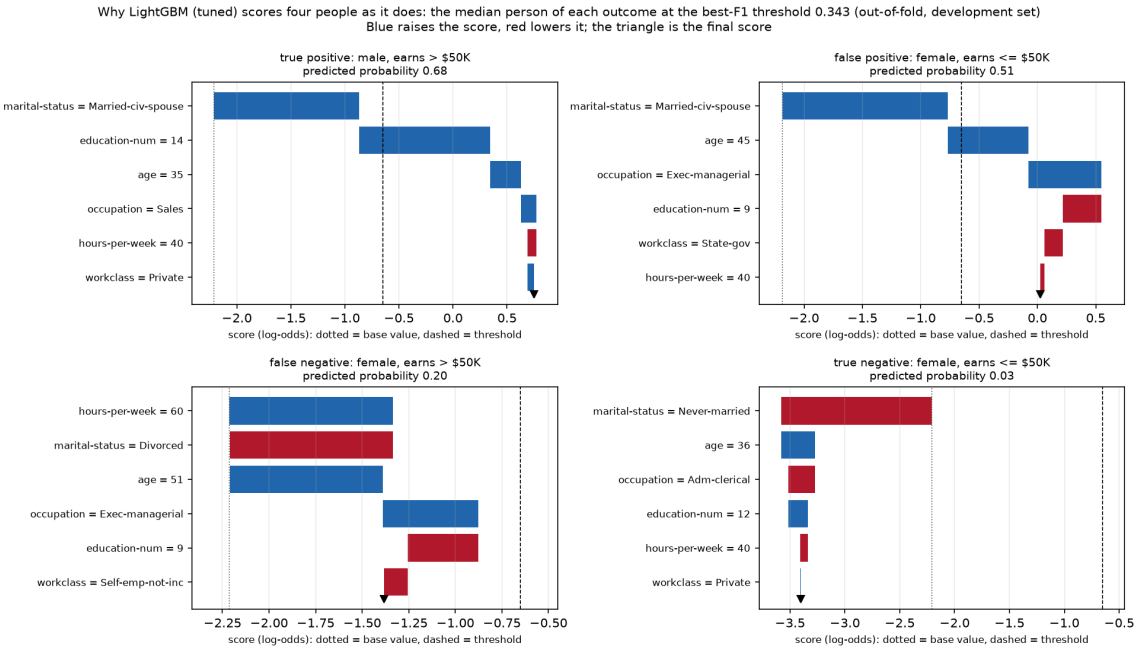


Figure 5: Local SHAP explanations of four people, chosen by rule (median score of each outcome at the 0.343 threshold), out-of-fold, development set. Each bar moves the score from the average (dotted) towards the final score (triangle); the dashed line is the threshold.

We did not remove marital status. It is the strongest legitimate signal (married women and men earn alike), and the other inputs differ by sex too, so removing it would partly move the proxy effect to them. We handle the bias at the decision step instead.

8 Fairness audit and mitigation

Metrics. For group g (women, men), decision $\hat{Y}$ and outcome Y, the selection rate is $\mathrm{SR}_g = P(\hat{Y} = 1 \mid g)$, the true-positive rate $\mathrm{TPR}_g = P(\hat{Y} = 1 \mid Y = 1, g)$, the false-positive rate $\mathrm{FPR}_g = P(\hat{Y} = 1 \mid Y = 0, g)$ and the precision $\mathrm{PPV}_g = P(Y = 1 \mid \hat{Y} = 1, g)$. We report

$$\text{EO gap} = \max\left(|\mathrm{TPR}_w - \mathrm{TPR}_m|,\ |\mathrm{FPR}_w - \mathrm{FPR}_m|\right), \qquad \text{four-fifths ratio} = \frac{\mathrm{SR}_w}{\mathrm{SR}_m}\ \ (\text{screen: } \geq 0.8). \tag{6}$$

Equalized odds asks for an EO gap of 0, demographic parity for equal selection rates and predictive parity for equal precision. Intervals come from bootstrap resamples of test people.

Before mitigation. At the single threshold of 0.343, the final model finds 57.0% of high-earning women against 78.5% of high-earning men (TPR gap -0.215, 95% interval $[-0.272, -0.164]$). It wrongly flags 5.4% of lower-earning women against 22.0% of men, and selects 11.0% of women against 39.2% of men, a four-fifths ratio of 0.282 $[0.253, 0.309]$. A flag means almost the same for both (precision 0.564 against 0.610). So equalized odds and the four-fifths rule fail while predictive parity roughly holds, the same picture as on the development set (TPR gap -0.194, ratio 0.274). The base rates differ (10.9% of women and 30.5% of men in the test set), and the model reproduces this gap through its proxies (Section 7).

Mitigation. We applied one post-processing technique, Fairlearn's `ThresholdOptimizer` [13], which picks one threshold per sex (randomising between two thresholds where needed) to maximise balanced accuracy under an equalized-odds constraint. It was fitted on the development out-of-fold scores (after Platt scaling, which changes nothing for this model, Section 6) and applied once to the test set. We compare it with the single threshold that maximises the same objective (0.208).

Decision rule (test set)	Bal. acc.	F1	TPR gap	FPR gap	EO gap	PPV gap	Four-fifths
One threshold (best F1, 0.343)	0.799	0.670	-0.215	-0.166	0.215	-0.046	0.282
One threshold (best bal. acc., 0.208)	0.814	0.659	-0.140	-0.222	0.222	-0.082	0.356
Equalized odds (per-sex thresholds)	0.785	0.627	$+0.004$	$+0.003$	**0.004**	-0.304	0.745
Equal opportunity (per-sex thresholds)	0.809	0.658	$+0.003$	-0.111	0.111	-0.183	0.528
Demographic parity (per-sex thresholds)	0.775	0.614	$+0.137$	$+0.100$	0.137	-0.349	0.983

Table 3: Fairness by sex on the test set (gaps are women minus men), with every rule fixed on the development set. Equalized odds is our chosen mitigation; the last two rows are a sensitivity check.

After mitigation. Equalized odds closes both error-rate gaps on new people: TPR gap $+0.004$ $[-0.042, +0.047]$ and FPR gap $+0.003$ $[-0.017, +0.024]$ (Table 3). It gets there by flagging more women (selection 17.9% to 31.6%, FPR 10.8% to 25.4%) and fewer men (50.2% to 42.5%, FPR 33.1% to 25.1%) (Figure 6). The cost is 0.029 of balanced accuracy (0.814 to 0.785), with precision falling from 0.530 to 0.507 and recall from 0.872 to 0.822. The four-fifths ratio rises from 0.356 to 0.745 $[0.702, 0.793]$ but still fails, and a flag now means much less for a woman than for a man (precision 0.285 against 0.589).

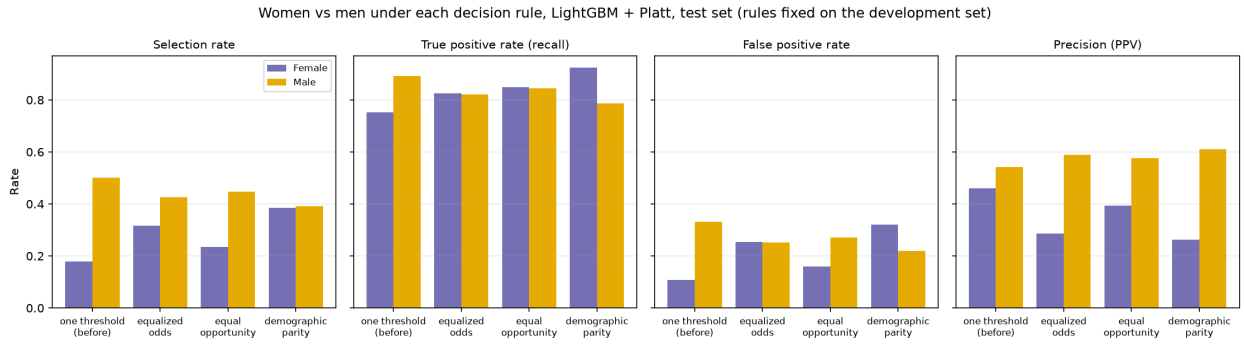


Figure 6: Women and men under each decision rule, test set. "One threshold (before)" is the single threshold that maximises balanced accuracy (0.208), the same objective as the per-sex thresholds.

The definitions conflict. Enforcing equal opportunity instead keeps the TPR gap at $+0.003$ but leaves an FPR gap of -0.111. Demographic parity passes the four-fifths screen (0.983) but opens a TPR gap of $+0.137$ and a precision gap of -0.349. When base rates differ, no rule can equalise selection rates, error rates and precision at once [8]. We keep equalized odds, as decided before the audit, and state plainly that it fails the legal screen.

Race. At the same threshold, Black people are selected less often than White people (13.2% against 31.6%), with a lower TPR (0.722 against 0.754) and FPR (0.061 against 0.167), for base rates of 10.8% and 25.3%. The other groups have between 80 and 311 people in the test set, too few for reliable rates. Only sex was mitigated.

9 Limitations and conclusion

The data is a 1994 US snapshot with sex recorded as binary and an arbitrary \$50K threshold, which limits its value as a fairness benchmark [4]. Six legitimate inputs carry about 0.71 PR-AUC; we gave up stronger but leaking or sex-revealing columns on purpose. Cross-validated scores of tuned models are optimistic (0.0035 PR-AUC here); nested cross-validation would have measured this without the test set. One test set of 9,758 people leaves an uncertainty of about ± 0.02 PR-AUC per model, which is why we compared models with paired intervals. The mitigation needs sex at decision time and randomises some decisions, which some legal contexts do not allow; an in-processing method that needs sex only for training is the next thing to test, along with client-specific thresholds.

Our model is as good as its inputs allow and its probabilities can be trusted for women and men. It still treats them differently, and the reason is visible: marital status and hours, two proxies for sex, carry most of the score gap. Post-processing can equalise error rates, but it costs accuracy, makes a flag less informative for women and still fails the four-fifths rule. Which definition of fairness to enforce is a policy choice for the client, and an audit should report all of them.

Contribution statement

Member	Contributions
Manel Rahali	GitHub repository and branch workflow; exploratory analysis (integrity, types, outliers, rare categories, features against target, leakage audit, cleaning and split); `src/data.py`; preprocessing in `src/pipeline.py`; project architecture; random forest diagnosis, validation curves and tuning v2; README; first draft of the report.
Soumaya Atoui	Business case and target orientation; feature engineering; analysis of groups and proxies; baseline progression; Optuna tuning v1 and MLflow tracking; first calibration and fairness audit scripts; findings log; logistic regression diagnosis (validation curves, Lasso and Elastic Net, interactions); local SHAP explanations; final report.
Hadiqa Javaid	LightGBM diagnosis, validation curves and tuning v2; logistic regression tuning v2; calibration and fairness audit of the v2 models; global SHAP analysis; final evaluation on the test set.

Use of AI coding assistants. We used Claude (Anthropic) as an assistant to discuss methods, to draft and refactor code (project architecture, diagnosis, tuning and evaluation scripts) and to draft parts of this report from our findings log. Four commits in the repository were made under the assistant's name. Every script was run by us, every number comes from our runs, and every methodological choice was reviewed by the team.

References

[1] T. Akiba, S. Sano, T. Yanase, T. Ohta, M. Koyama. Optuna: a next-generation hyperparameter optimization framework. *KDD*, 2019.

[2] P. Besse, E. del Barrio, P. Gordaliza, J.-M. Loubes, L. Risser. A survey of bias in machine learning through the prism of statistical parity for the Adult data set. arXiv:2003.14263, 2020.

[3] N. Chakrabarty, S. Biswas. A statistical approach to adult census income level prediction. arXiv:1810.10076, 2018.

[4] F. Ding, M. Hardt, J. Miller, L. Schmidt. Retiring Adult: new datasets for fair machine learning. *NeurIPS*, 2021.

[5] US Equal Employment Opportunity Commission. Uniform Guidelines on Employee Selection Procedures, 29 CFR §1607.4(D), 1978.

[6] M. Hardt, E. Price, N. Srebro. Equality of opportunity in supervised learning. *NeurIPS*, 2016.

[7] G. Ke et al. LightGBM: a highly efficient gradient boosting decision tree. *NeurIPS*, 2017.

[8] J. Kleinberg, S. Mullainathan, M. Raghavan. Inherent trade-offs in the fair determination of risk scores. *ITCS*, 2017.

[9] R. Kohavi. Scaling up the accuracy of naive-Bayes classifiers: a decision-tree hybrid. *KDD*, 1996.

[10] S. M. Lundberg, S.-I. Lee. A unified approach to interpreting model predictions. *NeurIPS*, 2017.

[11] F. Pedregosa et al. Scikit-learn: machine learning in Python. *JMLR* 12, 2011.

[12] J. Platt. Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. *Advances in Large Margin Classifiers*, 1999.

[13] H. Weerts, M. Dudík, R. Edgar, A. Jalali, R. Lutz, M. Madaio. Fairlearn: assessing and improving fairness of AI systems. *JMLR* 24, 2023.

[14] M. Zaharia et al. Accelerating the machine learning lifecycle with MLflow. *IEEE Data Engineering Bulletin* 41(4), 2018.

A Experiment tracking

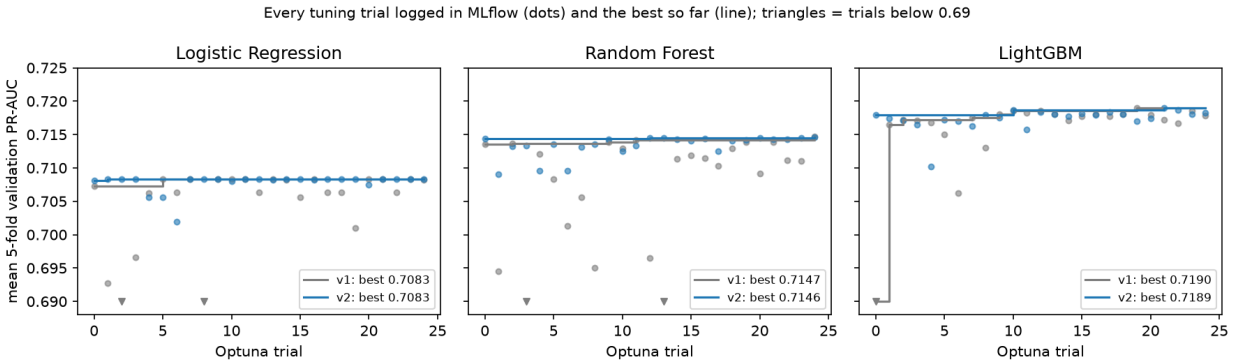


Figure 7: Every Optuna trial logged in MLflow (dots) and the best score so far (lines), tunings v1 (grey) and v2 (blue). Source: `python -m tuning.mlflow_history`, which reads the tracking store.

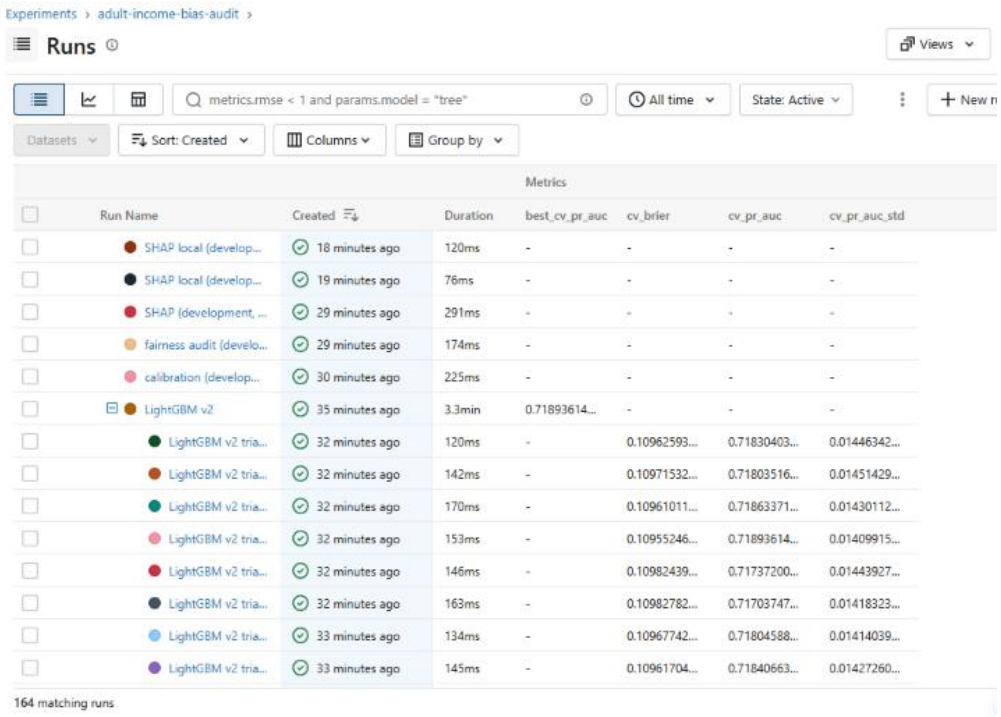


Figure 8: The MLflow runs table of experiment `adult-income-bias-audit` (164 runs). The LightGBM v2 study (parent run, best CV PR-AUC 0.7189) is expanded to show its trial runs, each with its mean 5-fold CV PR-AUC, the standard deviation over the folds and the Brier score; the evaluation runs (calibration, fairness, SHAP) sit above it.

B Tuned hyperparameters (v2)

Model	Best v2 settings (`tuning/results/v2/best_params.json`)
Logistic regression	$C = 6.80$, L2 penalty, no class weights (searched: C from 0.01 to 100, log scale).
Random forest	300 trees, Gini, no class weights; 6 people per leaf, depth 16, 10 people to split, 39% of people and $\sqrt{37}$ columns per tree.
LightGBM	950 trees, learning rate 0.014, 12 leaves, depth 10, 46 people per leaf, 80% of people and 43% of columns per tree, L1 1.56, L2 0.74, no class weight.

C Reproducing the results

From the project root, with the environment of `requirements.txt`: `python -m tuning.baseline`; `python -m tuning.<model> baseline` and `curves` for `logistic_regression`, `random_forest` and `lightgbm`; `python -m tuning.tune -version v2`; `python -m src.evaluate calibration`; `python -m src.fairness audit`; `python -m src.evaluate shap`; `python -m src.shap_local`; `python -m src.evaluate test` (run once); `python -m tuning.mlflow_history`; `python -m report.make_figures`.